

How to Upgrade to React 18

March 08, 2022 by [Rick Hanlon](#)

As we shared in the [release post](#), React 18 introduces features powered by our new concurrent renderer, with a gradual adoption strategy for existing applications. In this post, we will guide you through the steps for upgrading to React 18.

Please [report any issues](#) you encounter while upgrading to React 18.

Note

For React Native users, React 18 will ship in a future version of React Native. This is because React 18 relies on the New React Native Architecture to benefit from the new capabilities presented in this blogpost. For more information, see the [React Conf keynote here](#).

Installing

To install the latest version of React:

```
npm install react react-dom
```

Or if you're using yarn:

```
yarn add react react-dom
```

Updates to Client Rendering APIs

When you first install React 18, you will see a warning in the console:

Console

- `ReactDOM.render` is no longer supported in React 18. Use `createRoot` instead. Until you switch to the new API, your app will behave as if it's running React 17. Learn more: <https://reactjs.org/link/switch-to-createroot>

React 18 introduces a new root API which provides better ergonomics for managing roots. The new root API also enables the new concurrent renderer, which allows you to opt-into concurrent features.

```
// Before
import { render } from 'react-dom';
const container = document.getElementById('app');
render(<App tab="home" />, container);

// After
import { createRoot } from 'react-dom/client';
const container = document.getElementById('app');
const root = createRoot(container); // createRoot(container!) if you use TypeScript
root.render(<App tab="home" />);
```

We've also changed `unmountComponentAtNode` to `root.unmount`:

```
// Before
ReactDOM.unmountComponentAtNode(container);

// After
root.unmount();
```

We've also removed the callback from render, since it usually does not have the expected result when using Suspense:

```
// Before
const container = document.getElementById('app');
render(<App tab="home" />, container, () => {
  console.log('rendered');
});

// After
function AppWithCallbackAfterRender() {
  useEffect(() => {
    console.log('rendered');
  });

  return <App tab="home" />
}

const container = document.getElementById('app');
const root = createRoot(container);
root.render(<AppWithCallbackAfterRender />);
```

Note

There is no one-to-one replacement for the old render callback API — it depends on your use case. See the working group post for [Replacing](#)

[render with createRoot](#) for more information.

Finally, if your app uses server-side rendering with hydration, upgrade `hydrate` to `hydrateRoot`:

```
// Before
import { hydrate } from 'react-dom';
const container = document.getElementById('app');
hydrate(<App tab="home" />, container);

// After
import { hydrateRoot } from 'react-dom/client';
const container = document.getElementById('app');
const root = hydrateRoot(container, <App tab="home" />);
// Unlike with createRoot, you don't need a separate root.render() call here.
```

For more information, see the [working group discussion here](#).

Note

If your app doesn't work after upgrading, check whether it's wrapped in `<StrictMode>`. [Strict Mode has gotten stricter in React 18](#), and not all your components may be resilient to the new checks it adds in development mode. If removing Strict Mode fixes your app, you can remove it during the upgrade, and then add it back (either at the top or for a part of the tree) after you fix the issues that it's pointing out.

Updates to Server Rendering APIs

In this release, we're revamping our `react-dom/server` APIs to fully support Suspense on the server and Streaming SSR. As part of these changes, we're deprecating the old Node streaming API, which does not support incremental Suspense streaming on the server.

Using this API will now warn:

- `renderToNodeStream`: **Deprecated** 🛑

Instead, for streaming in Node environments, use:

- `renderToPipeableStream`: **New** ✨

We're also introducing a new API to support streaming SSR with Suspense for modern edge runtime environments, such as Deno and Cloudflare workers:

- `renderToReadableStream`: **New** ✨

The following APIs will continue working, but with limited support for Suspense:

- `renderToString`: **Limited** ⚠️
- `renderToStaticMarkup`: **Limited** ⚠️

Finally, this API will continue to work for rendering e-mails:

- `renderToStaticNodeStream`

For more information on the changes to server rendering APIs, see the working group post on [Upgrading to React 18 on the server](#), a [deep dive on the new Suspense SSR Architecture](#), and [Shaundai Person's](#) talk on [Streaming Server Rendering with Suspense](#) at React Conf 2021.

Updates to TypeScript definitions

If your project uses TypeScript, you will need to update your `@types/react` and `@types/react-dom` dependencies to the latest versions. The new types are safer and catch issues that used to be ignored by the type checker. The most

notable change is that the `children` prop now needs to be listed explicitly when defining props, for example:

```
interface MyButtonProps {  
  color: string;  
  children?: React.ReactNode;  
}
```

See the [React 18 typings pull request](#) for a full list of type-only changes. It links to example fixes in library types so you can see how to adjust your code. You can use the [automated migration script](#) to help port your application code to the new and safer typings faster.

If you find a bug in the typings, please [file an issue](#) in the DefinitelyTyped repo.

Automatic Batching

React 18 adds out-of-the-box performance improvements by doing more batching by default. Batching is when React groups multiple state updates into a single re-render for better performance. Before React 18, we only batched updates inside React event handlers. Updates inside of promises, `setTimeout`, native event handlers, or any other event were not batched in React by default:

```
// Before React 18 only React events were batched  
  
function handleClick() {  
  setCount(c => c + 1);  
  setFlag(f => !f);  
  // React will only re-render once at the end (that's batching!)  
}  
  
setTimeout(() => {  
  setCount(c => c + 1);  
  setFlag(f => !f);
```

```
// React will render twice, once for each state update (no batching)
}, 1000);
```

Starting in React 18 with `createRoot`, all updates will be automatically batched, no matter where they originate from. This means that updates inside of timeouts, promises, native event handlers or any other event will batch the same way as updates inside of React events:

```
// After React 18 updates inside of timeouts, promises,
// native event handlers or any other event are batched.

function handleClick() {
  setCount(c => c + 1);
  setFlag(f => !f);
  // React will only re-render once at the end (that's batching!)
}

setTimeout(() => {
  setCount(c => c + 1);
  setFlag(f => !f);
  // React will only re-render once at the end (that's batching!)
}, 1000);
```

This is a breaking change, but we expect this to result in less work rendering, and therefore better performance in your applications. To opt-out of automatic batching, you can use `flushSync`:

```
import { flushSync } from 'react-dom';

function handleClick() {
  flushSync(() => {
    setCounter(c => c + 1);
  });
  // React has updated the DOM by now
```

```
flushSync(() => {  
  setFlag(f => !f);  
});  
// React has updated the DOM by now  
}
```

For more information, see the [Automatic batching deep dive](#).

New APIs for Libraries

In the React 18 Working Group we worked with library maintainers to create new APIs needed to support concurrent rendering for use cases specific to their use case in areas like styles, and external stores. To support React 18, some libraries may need to switch to one of the following APIs:

- `useSyncExternalStore` is a new Hook that allows external stores to support concurrent reads by forcing updates to the store to be synchronous. This new API is recommended for any library that integrates with state external to React. For more information, see the [useSyncExternalStore overview post](#) and [useSyncExternalStore API details](#).
- `useInsertionEffect` is a new Hook that allows CSS-in-JS libraries to address performance issues of injecting styles in render. Unless you've already built a CSS-in-JS library we don't expect you to ever use this. This Hook will run after the DOM is mutated, but before layout effects read the new layout. This solves an issue that already exists in React 17 and below, but is even more important in React 18 because React yields to the browser during concurrent rendering, giving it a chance to recalculate layout. For more information, see the [Library Upgrade Guide for <style>](#).

React 18 also introduces new APIs for concurrent rendering such as `startTransition`, `useDeferredValue` and `useId`, which we share more about in the [release post](#).

Updates to Strict Mode

In the future, we'd like to add a feature that allows React to add and remove sections of the UI while preserving state. For example, when a user tabs away from a screen and back, React should be able to immediately show the previous screen. To do this, React would unmount and remount trees using the same component state as before.

This feature will give React better performance out-of-the-box, but requires components to be resilient to effects being mounted and destroyed multiple times. Most effects will work without any changes, but some effects assume they are only mounted or destroyed once.

To help surface these issues, React 18 introduces a new development-only check to Strict Mode. This new check will automatically unmount and remount every component, whenever a component mounts for the first time, restoring the previous state on the second mount.

Before this change, React would mount the component and create the effects:

- * React mounts the component.
 - * Layout effects are created.
 - * Effect effects are created.

With Strict Mode in React 18, React will simulate unmounting and remounting the component in development mode:

- * React mounts the component.
 - * Layout effects are created.
 - * Effect effects are created.
- * React simulates unmounting the component.
 - * Layout effects are destroyed.
 - * Effects are destroyed.
- * React simulates mounting the component **with** the previous state.
 - * Layout effect setup code runs

For more information, see the Working Group posts for [Adding Reusable State to StrictMode](#) and [How to support Reusable State in Effects](#).

Configuring Your Testing Environment

When you first update your tests to use `createRoot`, you may see this warning in your test console:



To fix this, set `globalThis.IS_REACT_ACT_ENVIRONMENT` to `true` before running your test:

```
// In your test setup file
globalThis.IS_REACT_ACT_ENVIRONMENT = true;
```

The purpose of the flag is to tell React that it's running in a unit test-like environment. React will log helpful warnings if you forget to wrap an update with `act`.

You can also set the flag to `false` to tell React that `act` isn't needed. This can be useful for end-to-end tests that simulate a full browser environment.

Eventually, we expect testing libraries will configure this for you automatically. For example, the [next version of React Testing Library has built-in support for React 18](#) without any additional configuration.

[More background on the `act` testing API and related changes](#) is available in the working group.

Dropping Support for Internet Explorer

In this release, React is dropping support for Internet Explorer, which is [going out of support on June 15, 2022](#). We're making this change now because new features introduced in React 18 are built using modern browser features such as microtasks which cannot be adequately polyfilled in IE.

If you need to support Internet Explorer we recommend you stay with React 17.

Deprecations

- `react-dom: ReactDOM.render` has been deprecated. Using it will warn and run your app in React 17 mode.
- `react-dom: ReactDOM.hydrate` has been deprecated. Using it will warn and run your app in React 17 mode.
- `react-dom: ReactDOM.unmountComponentAtNode` has been deprecated.
- `react-dom: ReactDOM.renderSubtreeIntoContainer` has been deprecated.
- `react-dom/server: ReactDOMServer.renderToNodeStream` has been deprecated.

Other Breaking Changes

- **Consistent `useEffect` timing:** React now always synchronously flushes effect functions if the update was triggered during a discrete user input event such as a click or a keydown event. Previously, the behavior wasn't always predictable or consistent.
- **Stricter hydration errors:** Hydration mismatches due to missing or extra text content are now treated like errors instead of warnings. React will no longer attempt to "patch up" individual nodes by inserting or deleting a node on the client in an attempt to match the server markup, and will revert to client rendering up to the closest `<Suspense>` boundary in the tree. This ensures the hydrated tree is consistent and avoids potential privacy and security holes that can be caused by hydration mismatches.

- **Suspense trees are always consistent:** If a component suspends before it's fully added to the tree, React will not add it to the tree in an incomplete state or fire its effects. Instead, React will throw away the new tree completely, wait for the asynchronous operation to finish, and then retry rendering again from scratch. React will render the retry attempt concurrently, and without blocking the browser.
- **Layout Effects with Suspense:** When a tree re-suspends and reverts to a fallback, React will now clean up layout effects, and then re-create them when the content inside the boundary is shown again. This fixes an issue which prevented component libraries from correctly measuring layout when used with Suspense.
- **New JS Environment Requirements:** React now depends on modern browsers features including `Promise`, `Symbol`, and `Object.assign`. If you support older browsers and devices such as Internet Explorer which do not provide modern browser features natively or have non-compliant implementations, consider including a global polyfill in your bundled application.

Other Notable Changes

React

- **Components can now render `undefined`:** React no longer warns if you return `undefined` from a component. This makes the allowed component return values consistent with values that are allowed in the middle of a component tree. We suggest to use a linter to prevent mistakes like forgetting a `return` statement before JSX.
- **In tests, `act` warnings are now opt-in:** If you're running end-to-end tests, the `act` warnings are unnecessary. We've introduced an [opt-in](#) mechanism so you can enable them only for unit tests where they are useful and beneficial.
- **No warning about `setState` on unmounted components:** Previously, React warned about memory leaks when you call `setState` on an unmounted component. This warning was added for subscriptions, but people primarily

run into it in scenarios where setting state is fine, and workarounds make the code worse. We've [removed](#) this warning.

- **No suppression of console logs:** When you use Strict Mode, React renders each component twice to help you find unexpected side effects. In React 17, we've suppressed console logs for one of the two renders to make the logs easier to read. In response to [community feedback](#) about this being confusing, we've removed the suppression. Instead, if you have React DevTools installed, the second log's renders will be displayed in grey, and there will be an option (off by default) to suppress them completely.
- **Improved memory usage:** React now cleans up more internal fields on unmount, making the impact from unfixed memory leaks that may exist in your application code less severe.

React DOM Server

- **`renderToString`:** Will no longer error when suspending on the server. Instead, it will emit the fallback HTML for the closest `<Suspense>` boundary and then retry rendering the same content on the client. It is still recommended that you switch to a streaming API like `renderToPipeableStream` or `renderToReadableStream` instead.
- **`renderToStaticMarkup`:** Will no longer error when suspending on the server. Instead, it will emit the fallback HTML for the closest `<Suspense>` boundary.

Changelog

You can view the [full changelog here](#).

PREVIOUS
< [React v18.0](#)

NEXT
>



Copyright © Meta Platforms, Inc

uwu?

Learn React

Quick Start

Installation

Describing the UI

Adding Interactivity

Managing State

Escape Hatches

API Reference

React APIs

React DOM APIs

Community

Code of Conduct

Meet the Team

Docs Contributors

Acknowledgements

More

Blog

React Native

Privacy

Terms

